

PDF Benchmark Tool

Metric Calculation Documentation

3-Stage Metric Framework for PDF-to-Audio Transformation

Version 1.0

Table of Contents

Overview

Stage 1: Extraction Efficiency (PDF to Text)

- 1.1 Layout F1-Score (Structural Integrity)
- 1.2 Word Error Rate (Textual Accuracy)
- 1.3 Extraction Latency (Speed)

Stage 2: TTS Efficiency (Text to Speech)

- 2.1 Mean Opinion Score (Perceptual Quality)
- 2.2 Real-Time Factor (Processing Speed)
- 2.3 WSD Accuracy (Word Sense Disambiguation)

Stage 3: System Health (The North Star)

- 3.1 System Health (Harmonic Mean)

Data Flow: How the Tool Runs a Benchmark

Custom PDF vs. Benchmark PDF: Differences

Quick Reference: All Metrics at a Glance

Overview

The PDF Benchmark Tool evaluates PDF extraction and text-to-speech pipelines using a three-stage metric framework. This document explains how each metric is calculated, what data it uses, what the scores mean, and how to interpret them.

Stage	Metrics	What It Evaluates
Stage 1: Extraction	Layout F1, WER, Latency	How accurately the PDF text is extracted
Stage 2: TTS	MOS, RTF, WSD	How well the text-to-speech engine performs
Stage 3: System	System Health (Harmonic Mean)	Overall pipeline reliability

Stage 1: Extraction Efficiency (PDF to Text)

Stage 1 measures how faithfully the extraction system reconstructs the document's content. It answers three questions: Is the structure correct? Are the words correct? How fast is it?

1.1 Layout F1-Score (Structural Integrity)

What it measures

The precision and recall of the document's structural reconstruction. It checks whether headings, body content, and reading order are preserved, and whether noise (headers, footers, page numbers) is correctly filtered out.

How it works

The tool compares extracted text against known reference data across four categories:

A. Structural Elements (headings and titles). For the built-in benchmark PDF, it looks for 6 specific elements: the title ("the art of digital reading") and 5 chapter/section headings. For custom PDFs, the tool auto-detects headings by looking for patterns like "Chapter X", "Part X", "Section X"; lines in ALL CAPS with 2 to 10 words; or short lines (under 60% of average line length) that start with a capital letter, have 2 to 12 words, and do not end with punctuation.

B. Body Content (key phrases). For the benchmark PDF, it checks for 12 key phrases that should appear in the extracted text. For custom PDFs, it extracts the first 6 words of sentences with more than 8 words from the reference text and checks whether they show up in the extraction.

C. Noise Filtering. For the benchmark PDF, it checks if 5 specific noise strings are present: "benchmark document v1.0", "confidential", "page 1 of 3", "page 2 of 3", and "page 3 of 3". These are headers, footers, and page numbers that should have been removed. For custom PDFs, it searches for common noise patterns like "Page X", "Page X of Y", standalone numbers, "copyright", and "all rights reserved".

D. Reading Order. The tool walks through the expected structural elements in order and checks whether each one appears after the previous one in the extracted text. If the extractor rearranged content (for example, reading a sidebar in the middle of a paragraph), the order score drops.

The formula

True Positives (TP) = structural elements found + body phrases found + noise correctly filtered out

False Positives (FP) = noise that leaked through (present in output when it should not be)

False Negatives (FN) = structural elements missing + body phrases missing

$$\text{Precision} = \text{TP} / (\text{TP} + \text{FP})$$

$$\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$$

$$\text{F1} = (2 \times \text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$$

Worked example

Suppose the extractor finds 5 of 6 headings, 10 of 12 body phrases, and filters out 4 of 5 noise patterns (1 leaked through):

$$\text{TP} = 5 + 10 + 4 = 19$$

$$\text{FP} = 1 \text{ (noise that leaked)}$$

$$\text{FN} = (6-5) + (12-10) = 3 \text{ (missing content)}$$

$$\text{Precision} = 19 / (19+1) = 0.950$$

$$\text{Recall} = 19 / (19+3) = 0.864$$

$$\text{F1} = (2 \times 0.950 \times 0.864) / (0.950 + 0.864) = 0.905 = 90.5\%$$

Score interpretation

F1 Score	Rating	Meaning
95 to 100%	Excellent	Near-perfect structural reconstruction
80 to 95%	Good	Minor structural issues, most content preserved
50 to 80%	Acceptable	Noticeable missing content or noise leaking through
Below 50%	Poor	Major structural failures, content missing or disordered

1.2 Word Error Rate (Textual Accuracy)

What it measures

The percentage of words that differ between the reference text and the extracted text. It catches character-level extraction errors like "O N" instead of "On", "ef-ficient" instead of "efficient", or missing and extra words.

How it works

The tool uses the Levenshtein edit distance algorithm at the word level. Both texts are first normalized: converted to lowercase, all whitespace collapsed to single spaces, and special characters standardized. Then the algorithm counts the minimum number of word-level edits needed to transform the extracted text into the reference text.

For custom PDFs (partial reference): Since the user only pastes a few pages, the tool first locates where the pasted text appears inside the full extraction. It takes the first 6 to 8 words of the reference and searches for them in the extracted text. Once found, it creates a matching window (the reference length plus a 15% buffer) and runs WER only on that matched region.

The algorithm (Levenshtein Distance)

A matrix is built comparing every reference word against every extracted word. Each cell represents the minimum edits to transform that portion of text. Three operations, each costing 1 edit:

- Substitution: "on" was extracted as "o" (wrong word)
- Insertion: extra "n" appeared (extra word)
- Deletion: "the" was missing (missing word)

The formula

WER = Total Edits / Reference Word Count

Worked example

Reference: "on the final day of my sophomore year" (8 words)

Extracted: "o n the final day of my sophomore year" (9 words)

"on" -> "o" = 1 substitution

extra "n" = 1 insertion

Total edits = 2

WER = 2 / 8 = 0.25 = 25%

Score interpretation

WER	Rating	Meaning
0%	Perfect	Every word extracted correctly
1 to 3%	Excellent	Minor formatting artifacts

WER	Rating	Meaning
3 to 5%	Good	Some errors, mostly readable
5 to 10%	Acceptable	Noticeable errors
10%+	Poor	Significant extraction failures

1.3 Extraction Latency (Speed)

What it measures

The total time from uploading the PDF to receiving the extracted clean text, measured in milliseconds using the browser's `performance.now()` API.

How it works

```

t0 = performance.now()           // start timer
upload PDF to server              // POST /api/v1/documents/upload
wait for server to process        // extraction pipeline runs
receive response with doc ID      // server returns extracted data
t1 = performance.now()           // stop timer
latency = t1 - t0                 // total milliseconds

```

This includes network round-trip time, file upload time, and server-side extraction processing. It does not include the text fetch step, which is a separate call.

Score interpretation

Latency	Rating	Indicator
Under 1 second	Excellent	Green
1 to 3 seconds	Acceptable	Yellow
Over 3 seconds	Slow	Red

Stage 2: TTS Efficiency (Text to Speech)

Stage 2 measures the quality and speed of the text-to-speech engine. It evaluates how natural the generated speech sounds, how fast it is generated, and whether the extraction preserved context needed for correct pronunciation.

2.1 Mean Opinion Score (Perceptual Quality)

What it measures

How natural the generated speech sounds, on a scale of 1 to 5. In industry, MOS is typically measured by having humans rate audio clips. This tool uses an automated proxy based on timing analysis.

MOS Score	Quality	Description
4.5 to 5.0	Excellent	Sounds like a real human
4.0 to 4.5	Very Good	Natural with minor imperfections
3.0 to 4.0	Acceptable	Noticeable synthetic quality
2.0 to 3.0	Poor	Clearly robotic
1.0 to 2.0	Bad	Unintelligible or broken

Method 1: Word Boundary Analysis (preferred)

Used when the TTS engine returns word-level timing data (a `word_boundaries` array with 5 or more entries). This method analyzes three aspects of the speech timing:

Component A: Rhythm Score (40% weight)

Natural speech has varied timing between words. Nobody says every word at exactly the same speed. The tool measures this variation using the Coefficient of Variation (CV):

CV = Standard Deviation of word gaps / Mean of word gaps

Example calculation:

```
Word gaps (ms): [320, 360, 170, 290, 410, 200, 380]
Mean = 304ms
Std Dev = 93ms
CV = 93 / 304 = 0.31
```

CV Range	What It Sounds Like	Score
0.30 to 0.70	Natural variation, human-like rhythm	4.5
0.20 to 0.85	Slightly off but generally acceptable	3.8
0.10 to 1.00	Noticeably unnatural (too uniform or too erratic)	3.0
Outside all ranges	Broken or extremely robotic	2.0

A low CV (say 0.05) means every gap is nearly identical, which sounds like a robotic monotone. A high CV (say 1.5) means wildly inconsistent gaps, which sounds glitchy. Natural speech falls in the middle.

Component B: Pause Score (35% weight)

In natural speech, you pause longer after sentences and commas than between words mid-sentence. The tool checks every word ending with a period, exclamation mark, question mark, or comma. If the gap after it exceeds 1.3 times the average gap, that counts as a natural pause.

Example:

Average gap: 304ms

Threshold: $304 \times 1.3 = 395\text{ms}$

After "years." -> gap is 450ms -> $450 > 395$ -> natural pause -> +0.15

After "world." -> gap is 420ms -> $420 > 395$ -> natural pause -> +0.15

After "spans" -> not punctuation -> skip

The score starts at 3.0. Each natural pause adds 0.15, capped at 5.0. If the TTS engine treats periods the same as regular words (no extra pause), the score stays at 3.0.

Component C: Speed Score (25% weight)

Normal conversation speed is 120 to 180 words per minute. Audiobooks typically run at 150 to 160 WPM.

WPM = Total Words / (Audio Duration in seconds / 60)

WPM Range	What It Sounds Like	Score
120 to 180	Natural conversation speed	4.5
100 to 200	Slightly fast or slow	3.8
80 to 220	Uncomfortably fast or slow	3.0
Outside	Unusable	2.5

Final MOS Calculation (Method 1)

MOS = (Rhythm x 0.40) + (Pause x 0.35) + (Speed x 0.25)

Worked example:

Rhythm Score: 4.5 (CV = 0.31, natural variation)

Pause Score: 4.2 (8 of 10 punctuation marks have proper pauses)

Speed Score: 4.5 (WPM = 155, perfect range)

$\text{MOS} = (4.5 \times 0.40) + (4.2 \times 0.35) + (4.5 \times 0.25)$

$\text{MOS} = 1.80 + 1.47 + 1.125 = 4.4$

Method 2: Fallback Estimation

Used when word boundaries are missing or have fewer than 5 entries. This happens when the TTS engine (like Kokoro) does not return timing data. The fallback uses only two signals:

$$\text{MOS} = (\text{Speed Score} \times 0.60) + (\text{RTF Score} \times 0.40)$$

RTF Score mapping: RTF < 0.2 gives 4.5; RTF < 0.5 gives 3.8; RTF < 1.0 gives 3.0; anything higher gives 2.0.

This method is less accurate because it cannot measure rhythm or pause patterns. It typically scores 2.5 to 3.5 regardless of actual audio quality. To get accurate MOS with engines like Kokoro, the backend should provide word-level timing data or a forced alignment tool (like WhisperX) should be used to generate timestamps after synthesis.

2.2 Real-Time Factor (Processing Speed)

What it measures

How fast the TTS engine generates audio relative to the audio's duration. An RTF of 0.1 means 10 seconds of audio is generated in 1 second.

The formula

$$\text{RTF} = \text{Synthesis Time} / \text{Audio Duration}$$

Both values come directly from the TTS API response. When the tool calls POST `/api/v1/documents/{id}/tts/segment/0`, the response includes "synthesis_time" (how long the server took) and "duration_seconds" (how long the audio is).

Worked example

```
synthesis_time = 48.5 seconds (server processing)
duration_seconds = 583.1 seconds (audio length)
RTF = 48.5 / 583.1 = 0.083
Meaning: 10 minutes of audio generated in 50 seconds
```

Score interpretation

RTF	Rating	What It Means
< 0.2	Excellent	5 min audio in under 1 min, scales well
0.2 to 0.5	Good	Acceptable for most use cases
0.5 to 1.0	Poor	Audio takes almost as long to generate as to listen
> 1.0	Critical	Slower than real-time, cannot scale

Target for a competitive product: RTF < 0.2. That means multiple concurrent users can be served without queuing.

2.3 WSD Accuracy (Word Sense Disambiguation)

What it measures

Whether the extraction preserves the context that TTS needs to pronounce heteronyms correctly. Heteronyms are words spelled the same but pronounced differently based on meaning:

```
"I read books daily"          (present tense: /reed/)
"I read that book yesterday"  (past tense: /red/)

"the tear in the fabric"      (rip: /tare/)
"a tear rolled down her cheek" (crying: /teer/)
```

How it works

For the benchmark PDF: The benchmark PDF contains known sentences with "read" and "tear" used in different contexts. The tool checks if both context sentences for each heteronym appear intact in the extracted text.

For custom PDFs: The tool scans the reference text for 40 common heteronyms (read, lead, tear, wind, bow, close, live, minute, object, present, record, refuse, desert, produce, project, subject, and others). For each heteronym found in multiple sentences, it checks whether those sentences survived extraction.

The formula

WSD = Contexts Found in Extraction / Total Contexts in Reference

Worked example

```
Heteronym "read":
  context1 "i read books daily" found      -> yes
  context2 "i read that book yesterday" found -> yes

Heteronym "tear":
  context1 "the tear in the fabric" found    -> yes
  context2 "a tear rolled down her cheek" found -> yes

WSD = 4/4 = 100%
```

Stage 3: System Health (The North Star)

Stage 3 produces a single number that represents overall pipeline health. It uses the harmonic mean, which heavily punishes weak links.

3.1 System Health (Harmonic Mean)

The formula

$$\text{System Health} = n / (1/x_1 + 1/x_2 + 1/x_3 + \dots + 1/x_n)$$

The three components

All scores are normalized to a 0 to 1 scale before computing the harmonic mean:

Component	How It Is Normalized	Source
Extraction Accuracy	1 - WER (e.g., WER 2.89% becomes 0.971)	Stage 1 WER
TTS Quality	MOS / 5 (e.g., MOS 3.3 becomes 0.66)	Stage 2 MOS
Success Rate	1.0 if F1 > 0, else 0.5	Stage 1 Layout F1

Why harmonic mean instead of a simple average?

The harmonic mean punishes outliers. If one component is terrible, the whole score drops dramatically. This reflects reality: a PDF reader with amazing extraction but a 90% crash rate is useless.

Worked example: balanced system

Extraction Accuracy: 0.971 (WER 2.89%)

TTS Quality: 0.66 (MOS 3.3)

Success Rate: 1.0 (F1 > 0)

$$\text{Health} = 3 / (1/0.971 + 1/0.66 + 1/1.0)$$

$$\text{Health} = 3 / (1.030 + 1.515 + 1.000)$$

$$\text{Health} = 3 / 3.545 = 0.846 = 84.6\%$$

Worked example: one weak link

Extraction Accuracy: 0.90 (good)

TTS Quality: 0.90 (good)

Success Rate: 0.10 (crashes 90% of the time)

$$\text{Standard Average: } (0.90 + 0.90 + 0.10) / 3 = 0.63 \text{ (looks passable)}$$

$$\text{Harmonic Mean: } 3 / (1/0.90 + 1/0.90 + 1/0.10) = 3 / 12.22 = 0.245 \text{ (failing)}$$

The harmonic mean correctly identifies this as a failing system, while the simple average hides the problem.

Data Flow: How the Tool Runs a Benchmark

Here is the step-by-step sequence when you run a benchmark:

Step 1: Upload PDF. The tool uploads the benchmark PDF (embedded as base64) or a custom PDF to the backend via POST `/api/v1/documents/upload`. The timer starts with `performance.now()` before the upload and stops when the response arrives. This gives Extraction Latency.

Step 2: Fetch Extracted Text. The tool calls GET `/api/v1/documents/{id}/text` to retrieve the clean extracted text. This text is used for Layout F1, WER, and WSD calculations.

Step 3: Fetch Processing Stats. The tool calls GET `/api/v1/documents/{id}/stats` to get metadata about the extraction (blocks extracted, headers removed, etc.). This is displayed in the detail view.

Step 4: Compute Extraction Metrics. Layout F1 is computed by comparing the extracted text against the reference. WER is computed using Levenshtein distance at the word level. Both computations happen entirely in the browser, no server calls needed.

Step 5: TTS Evaluation (if enabled). The tool calls GET `/api/v1/documents/{id}/tts/info` to check how many audio segments are available, then calls POST `/api/v1/documents/{id}/tts/segment/0` to generate the first segment. The response includes `synthesis_time`, `duration_seconds`, and `word_boundaries`. RTF is computed from synthesis time divided by audio duration. MOS is computed from the word boundary timing analysis (or the fallback estimation if boundaries are missing).

Step 6: WSD Check. The extracted text is searched for heteronym context sentences. For the benchmark PDF, this uses hardcoded sentence pairs. For custom PDFs, it scans the reference text for common heteronyms and their surrounding sentences.

Step 7: System Health. The three normalized component scores (Extraction Accuracy, TTS Quality, Success Rate) are combined using the harmonic mean formula.

Custom PDF vs. Benchmark PDF: Differences

Aspect	Benchmark PDF	Custom PDF
Reference Text	Hardcoded inside the tool (exact known content)	User pastes a few pages from the original source
Layout F1	Checks 6 exact headings + 12 body phrases + 5 noise strings	Auto-detects headings, extracts body phrases, checks common noise patterns
WER	Compares full extraction against full known reference	Finds matching region in extraction, compares only that window
WSD	Checks 2 hardcoded heteronym pairs (read, tear)	Scans pasted text for 40 common heteronyms
TTS Metrics	Same method	Same method

Aspect	Benchmark PDF	Custom PDF
System Health	Full harmonic mean of all 3 components	Based on available metrics only
Without reference	N/A (always has reference)	F1, WER, WSD show N/A; only Latency, RTF, MOS computed

Quick Reference: All Metrics at a Glance

Metric	Formula	Good Score	Data Source
Layout F1	$2 \times P \times R / (P+R)$	> 95%	Extracted text vs. reference structure
WER	Edits / Ref Words	< 3%	Extracted text vs. reference words
Latency	Upload end - start	< 1 second	Browser timer
MOS	$Rhythm(.4) + Pause(.35) + Speed(.25)$	> 4.0	TTS word boundaries
RTF	Synthesis Time / Audio Duration	< 0.2	TTS API response
WSD	Contexts Found / Total	100%	Extracted text vs. heteronym sentences
Health	Harmonic Mean of 3 scores	> 85%	Extraction + TTS + Success